Week 5: Automatic differentiation and checkpointing

Jin-Guo Liu, Xi-Wei Pan

Abstract: Each Part defines the terms this abstract uses. This week we learn to differentiate a time-stepping simulation with automatic differentiation and to run its reverse pass within a storage budget. We first write the forward and reverse passes of a pair energy by hand in JAX, compare them with the analytic derivative and with finite differences, and read the graphs JAX records. We then build an acoustic wave solver and differentiate it through time with Enzyme: forward mode gives the Born scattered data of a thin reflector, reverse mode back-propagates those data into an image that locates the reflector, and the transpose identity checks one against the other. Treeverse checkpointing then replaces the 241 saved states of the full trajectory by six while leaving the image unchanged, and the same operators image the dipping layers of a Marmousi model.

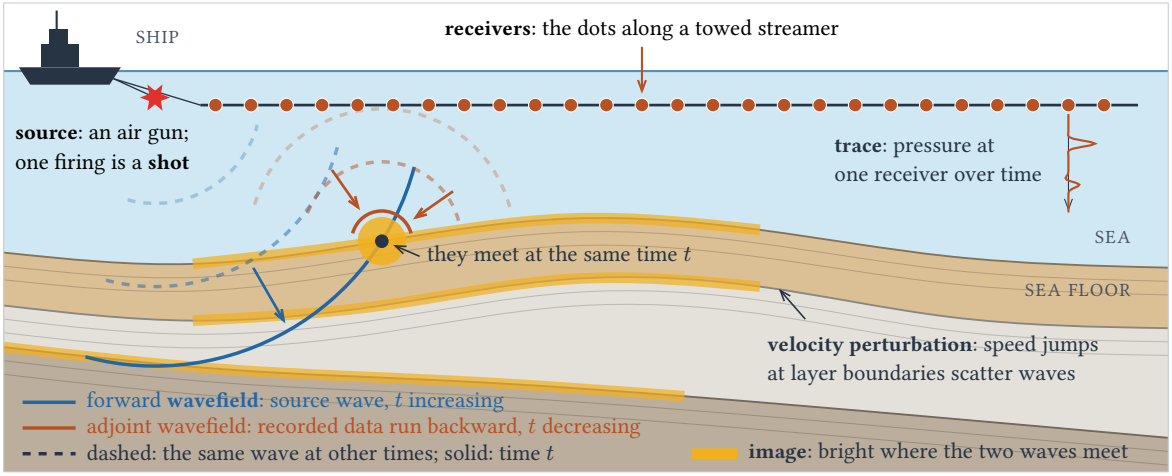
Resources:

- Liu and Xu, *Automatic differentiation and its applications in physics simulation* [1]. Parts 1, 3 and 4 identify the relevant reading.
- Symes, *Reverse time migration with optimal checkpointing* [2]. Parts 3 and 4 develop its modeling, adjoint calculation and imaging experiment.

The problem: imaging underground reflectors

Survey vocabulary. A source injects a pressure pulse into the ground. The resulting *pressure field* describes the wave amplitude at each position and time. A receiver samples pressure at a fixed position. One source activation is a shot. The pressure one receiver records over time is its trace. The traces of one shot form a shot gather. The background is the reference wave-speed field. A velocity perturbation is a change in that speed. It scatters the incident wave toward the receivers.

Reverse time migration. *Reverse time migration*, or *RTM*, turns recorded echoes into a picture of the reflectors underground. A forward run sends the source pulse into the background; its pressure field over time is the *wavefield*. A reverse run replays the scattered traces backward in time from the receivers. At each point, RTM combines the two fields into how much the data would change with the wave speed there, summed over time: the image. It is large where the two waves meet, at a reflector.



Reverse time migration in a marine survey. A ship tows an air-gun source and a streamer of receivers. RTM simulates the source wave forward in time and replays the recorded data backward from the receivers. The forward state supplies the local velocity derivative; the adjoint field weights its contribution to the image. The geology and wavefronts are drawn, not computed.

The acoustic wave equation. Write the pressure as $u(x, z, t)$, with horizontal position x, depth z and time t. Starting from rest, it obeys

$$\begin{cases} \partial_t^2 u = c^2 \nabla^2 u + q, & t > 0, \\ u = 0, \quad \partial_t u = 0, & t = 0, \end{cases} \tag{1}$$

where $c(x, z)$ is the wave speed, $q(x, z, t)$ is the source, and the *Laplacian* $\nabla^2 u = \partial_x^2 u + \partial_z^2 u$ measures the field's spatial curvature.

The discretized wave equation. A computer stores u on a grid. The *grid spacing* Δx is the distance between neighboring points, and the timestep Δt is the interval between successive time levels. Write $u_{i,j}^n$ for the pressure at grid point (i, j) and time level n. The five-point spatial Laplacian approximates $\nabla^2 u$ by

$$(L_h u)_{i,j} = \frac{u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1} - 4u_{i,j}}{\Delta x^2}. \tag{2}$$

Centered time differences use the levels on both sides of n: $\partial_t^2 u \approx (u^{n+1} - 2u^n + u^{n-1})/\Delta t^2$. Substituting both into Equation 1 and solving for u^{n+1} gives the numerical scheme

$$\begin{cases} u_{i,j}^{n+1} = 2u_{i,j}^n - u_{i,j}^{n-1} + \Delta t^2 \left(c_{i,j}^2 (L_h u^n)_{i,j} + q_{i,j}^n \right), & n = 0, 1, \ldots, N-1, \\ u_{i,j}^0 = u_{i,j}^{-1} = 0, & \\ y_k^n = u_{i_k, j_k}^{n+1}, & \text{receiver } k \text{ at grid point } (i_k, j_k), \end{cases} \tag{3}$$

for N time steps, where $c_{i,j}$ and $q_{i,j}^n$ are the wave speed and the source at grid point (i, j) and time $t = n\Delta t$, and y_k^n is sample n of receiver k's trace. Each step needs the two previous fields, so the pair $s_n = (u^{n-1}, u^n)$ is a complete state: the step maps it to $s_{n+1} = (u^n, u^{n+1})$, and it suffices to restart the simulation at step n.

Imaging as an inverse problem. Given c and the source, solving Equation 1 predicts every trace: the *forward problem*. Imaging asks the reverse question: given the recorded traces, where does c change? This is an *inverse problem*: it infers the underground from its echoes. Write $F(c)$ for all receiver traces computed with speed c. At a known background c_0, let J be the derivative of F: the matrix whose entry in row i and column j is $\partial F_i / \partial c_j$, how trace sample i changes with the speed in cell j. Its *transpose* J^T swaps rows and columns. Scaling a small velocity perturbation m by a number α changes the data, up to terms of order α^2, by

$$F(c_0 + \alpha m) \approx F(c_0) + \alpha J m. \tag{4}$$

The Born approximation keeps this first-order response: the scattered data are $d = Jm$. RTM maps them back to the image

$$m_{\text{img}} = J^T d = J^T J m. \tag{5}$$

The image also gives a direction for fitting the data. Hold the recorded data d fixed and let $\tilde{m}$ be a trial velocity perturbation. The least-squares misfit $\Phi(\tilde{m}) = \frac{1}{2} \sum (J\tilde{m} - d)^2$ sums over all trace samples. It vanishes when $\tilde{m} = m$, the perturbation that generated d. Its gradient with respect to the trial perturbation is $\nabla\Phi(\tilde{m}) = J^T(J\tilde{m} - d)$. At the initial trial $\tilde{m} = 0$, $\nabla\Phi(0) = -J^T d = -m_{\text{img}}$. The image therefore points in the direction of the steepest initial decrease in the misfit. $\tilde{m}$ has one entry per grid cell and Φ is one number. Automatic differentiation computes the image: Part 1 introduces both modes, Part 3 computes d and m_{img} through the time steps, and Part 4 fits the reverse pass in memory by keeping only a few complete states. The source's *bandwidth*, the range of frequencies in its pulse, and the receiver coverage limit the image's resolution: it locates a reflector, while its amplitude and shape differ from m.

Part 1: Forward and reverse automatic differentiation

Goal: hand-written JAX forward and reverse passes whose derivatives of a pair energy have maximum errors below 10^{-12} over the sampled interval, smaller than the maximum finite-difference error, a recorded gradient graph in which two contributions to a shared value add, and a many-input gradient whose reverse-mode cost stays near one energy evaluation while the forward-mode cost grows in proportion to the number of inputs.

(1) DO

Work in `week5/` inside your existing `AMAT5315-2026Fall-Exercise` repository.

```
mkdir week5 && cd week5
```

Start with the session rules:

```
Work in `week5/`, and read every path I give relative to it. After completing each
task, commit the work. When anything is uncertain, ask me instead of guessing. Put
run, comparison and plotting scripts in `scripts/`. Draw plots with matplotlib and
save them as PNG files. Keep `inputs/` and `.npy` files out of git.
```

Differentiate the pair energy by hand in JAX, in both modes:

```
Differentiate the Lennard-Jones pair energy U(r) = 4(r^-12 - r^-6) at r = 1.3 in JAX
with 64-bit floats. Use the nodes a = r^-6, b = a^2, c = b - a and U = 4c. Write the
forward-mode and reverse-mode passes by hand, one node at a time. Save `artifacts/ad/
derivatives.json` with
- r, energy: the separation and U(r)
- tangents: the tangent of each node r, a, b, c, U, from forward mode
- adjoints: the adjoint of each node, from reverse mode
- jax_grad: the derivative from JAX's own gradient function, at r
```

Read on while the agent works.

(2) UNDERSTAND

Pair energy and force. Week 2 used the Lennard-Jones pair energy $U(r)$ for two atoms separated by r. In units where atom size and energy scale are one,

$$U(r) = 4(r^{-12} - r^{-6}), \quad U'(r) = 24(r^{-7} - 2r^{-13}). \tag{6}$$

The force is $-U'(r)$. We check the derivative $U'(r)$ itself.

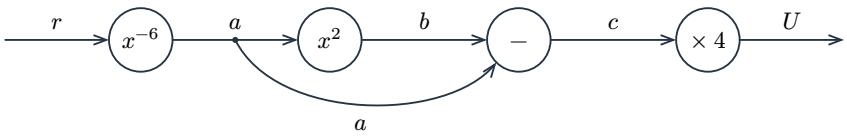
Finite differences. A finite difference estimates the derivative from nearby energies, with a small step h:

$$D_h U(r) = \frac{U(r+h) - U(r-h)}{2h}. \tag{7}$$

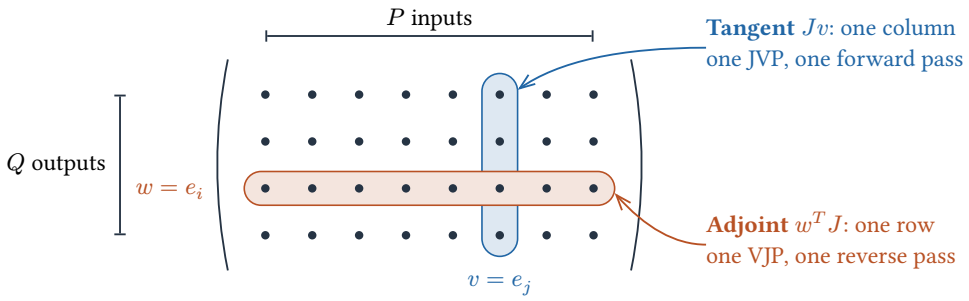
Its *truncation error* is proportional to h^2 for a smooth function. Its *cancellation error* grows as h shrinks: the two energies agree in more leading digits, so their difference keeps fewer of the 16 significant digits that a 64-bit number stores. This rounding error is near $10^{-16}/h$. The two errors balance where $h^3 \approx 10^{-16}$, so the best step lies between 10^{-6} and 10^{-5}.

The chain rule in a program. Automatic differentiation, or *AD*, applies the chain rule to the operations the program executes. The original calculation is the *primal*. AD differentiates that discrete calculation. Its remaining error comes from floating-point arithmetic alone.

The computational graph. A computational graph puts operations at nodes and the values they pass on edges. For the pair energy, write $a = r^{-6}$, $b = a^2$, $c = b - a$, and $U = 4c$. These a, b, c name Part 1's intermediates only; elsewhere c is the wave speed. A tangent describes the response to a change in the input; a dot marks it, as in $\dot{a} = \partial a/\partial r$ (not a time derivative here). Forward mode starts with $\dot{r} = 1$ and propagates these responses toward U. An adjoint measures how the output changes with an intermediate value; a bar marks it, as in $\overline{a} = \partial U/\partial a$. Reverse mode starts with $\overline{U} = 1$ and propagates backward.



Derivatives of many inputs and outputs. The opening's J is one example of a *Jacobian*: for any $f : \mathbb{R}^P \to \mathbb{R}^Q$, the Jacobian J holds the partial derivatives $\partial f_i/\partial x_j$, one row per output and one column per input. A *Jacobian-vector product*, or JVP, is Jv: the tangent of every output when the inputs move in direction v. One forward pass computes it. With $v = e_j$, the j-th unit vector, it is column j. Choose output weights w and form the scalar $w^T f$. Its gradient $J^T w$ contains one adjoint per input. The *vector-Jacobian product*, or VJP, is its row transpose $w^T J$, computed by one reverse pass. With $w = e_i$, it is row i of the Jacobian.



The whole Jacobian costs P forward passes or Q reverse passes. A scalar output has $Q = 1$, so one reverse pass gives its entire gradient. Part 3's image is one VJP over every grid cell.

The gradient of a cluster energy. The cluster energy of N atoms sums the pair energy over every pair, $E = \sum_{i<j} U(r_{ij})$, where r_{ij} is the distance between atoms i and j. It is one number that depends on $P = 3N$ coordinates, like a loss that depends on many parameters, and its gradient is minus the force on every atom. Forward mode needs P JVPs, one per coordinate; reverse mode needs one VJP. Dividing each gradient's time by one energy evaluation's time removes the speed of the machine. Reverse mode pays in memory instead: it keeps the $N(N-1)/2$ pair distances, which is small here. Part 4 meets a calculation whose kept values do not fit.

Files and tools. A *JSON* file stores named fields as text. *NumPy* is Python's array library; its `.npy` format stores one array in binary with its number type and shape, in a fraction of the space the same numbers take as text. *matplotlib* is Python's plotting library. *uv* manages the Python environment and pins package versions in `pyproject.toml` and `uv.lock`.

The graph an AD library records. JAX is a Python library for automatic differentiation; the agent writes the JAX code. Like other AD libraries, it runs a function once and records each operation it executes: the computational graph. JAX calls this record a *jaxpr*, one line per operation, and prints it with `jax.make_jaxpr`. Its gradient function, `jax.grad`, differentiates that record in reverse mode. JAX computes with 32-bit floats, which keep about seven digits, unless 64-bit floats are enabled. The checks below need at least twelve.

Forward mode by hand. Forward mode carries each value together with its tangent. At each node, the tangent of the result is one JVP: the node's partial derivatives times the tangents of its inputs. The pass runs once, from r to U, discarding each value and tangent after their last use; it needs no history for a later reverse sweep. The shared value a passes its tangent $\dot{a}$ to both users; no addition is needed at a. In JAX, `jax.jvp` of a node computes one line:

function LJ-Forward-Mode(r)
  $\dot{r} \leftarrow 1$ ▷ seed the input direction
  $a, \dot{a} \leftarrow r^{-6}, \quad (\partial a/\partial r)\dot{r}$ ▷ $\partial a/\partial r = -6r^{-7}$
  $b, \dot{b} \leftarrow a^2, \quad (\partial b/\partial a)\dot{a}$ ▷ $\partial b/\partial a = 2a$
  $c, \dot{c} \leftarrow b - a, \quad (\partial c/\partial b)\dot{b} + (\partial c/\partial a)\dot{a}$ ▷ $\dot{a}$ is used a second time
  $U, \dot{U} \leftarrow 4c, \quad (\partial U/\partial c)\dot{c}$ ▷ $\partial U/\partial c = 4$
  return $U, \dot{U}$
end

Reverse mode by hand. Reverse mode first runs the four nodes forward and keeps the primal values r and a that the local derivatives need. It then sweeps back from U to r. At each node, the adjoints of its inputs gain one VJP: the adjoint of the result times the node's partial derivatives. The value a feeds two nodes, so its adjoint is the sum of two terms. In JAX, `jax.vjp` of a node computes one such step:

function LJ-Reverse-Mode(r)
  $a, b, c, U \leftarrow r^{-6}, a^2, b - a, 4c$ ▷ forward sweep: keep r and a
  $\overline{U} \leftarrow 1$ ▷ seed the output weight
  $\overline{c} \leftarrow \overline{U}(\partial U/\partial c)$
  $\overline{b} \leftarrow \overline{c}(\partial c/\partial b)$
  $\overline{a} \leftarrow \overline{c}(\partial c/\partial a) + \overline{b}(\partial b/\partial a)$ ▷ a feeds two nodes: add both terms
  $\overline{r} \leftarrow \overline{a}(\partial a/\partial r)$
  return $U, \overline{r}$
end

Finite differences, forward mode and reverse mode compared. For $f : \mathbb{R}^P \to \mathbb{R}^Q$, let one evaluation of f need memory S, and let $\varepsilon \approx 10^{-16}$ be the rounding error of one 64-bit operation. $O(h^2)$ means proportional to h^2 for small h. The three methods trade accuracy, work and memory as follows [1]:

	Finite difference	**Forward mode**	**Reverse mode**
Accuracy	approximate: truncation error $O(h^2)$ plus rounding error $O(\varepsilon/h)$	exact up to rounding	exact up to rounding
Full Jacobian	$2P$ evaluations	P passes, one per input	Q passes, one per output
Memory	S	S: each value carries its tangent	grows with the steps: it keeps the primal values
Program	any black box	must be differentiable	must be differentiable
Suits	checking a derivative	few inputs, many outputs	many inputs, one scalar output

Reverse mode's memory is its cost. For the wave simulation it stores every complete state, which Part 4's checkpointing reduces.

Answer keys. The *answer keys* are the instructor's own implementations of this week's work. Printed expectations in VERIFY quote them: JAX for Part 1, Rust elsewhere. Reference figures come from an independent NumPy calculation, except Part 1's figures, drawn by the JAX answer key, and the two checkpoint figures of Part 4, which read the Rust run.

- Read Liu and Xu [1], sections 2 and 3. Find which primal values a reverse calculation needs to retain.

KEY CONCEPTS

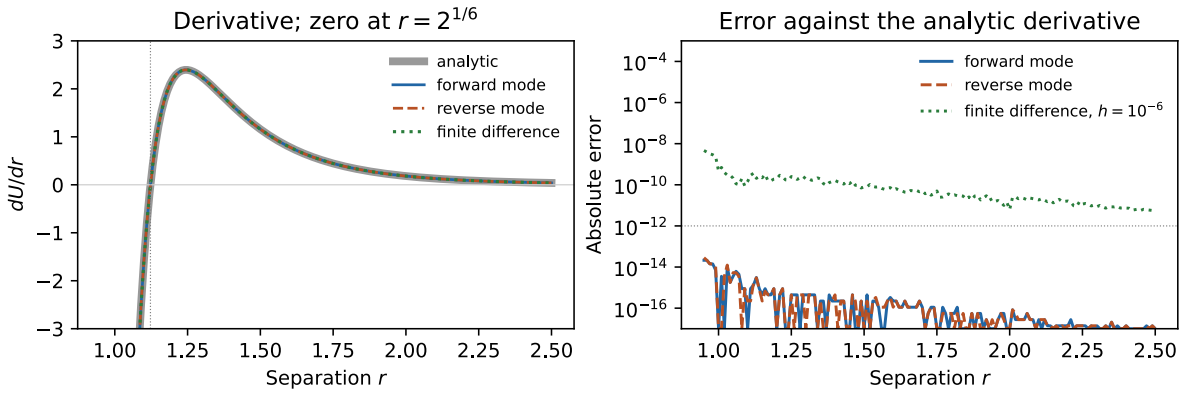
- A force is the negative derivative of pair energy.
- Finite differences balance truncation error against cancellation.
- Forward mode propagates tangents with JVPs; reverse mode propagates adjoints with VJPs and adds contributions to shared values.
- The whole Jacobian costs one forward pass per input or one reverse pass per output; a scalar gradient needs one reverse pass and its retained primal values.
- The gradient of a cluster energy costs a few energy evaluations in reverse mode and grows in proportion to its $3N$ inputs in forward mode.
- JAX records the computational graph; 64-bit floats keep about sixteen digits.
- **The check:** compare both hand-written modes and a finite difference with the analytic derivative, find the addition in JAX's gradient graph, then time a cluster-energy gradient in both modes.

3 VERIFY

(1) **Both AD modes have smaller maximum errors than finite differences.**

```
Compare your forward mode, reverse mode and centered finite difference with
analytic dU/dr. Draw two plots in reduced units: 1. x-axis is separation r, y-axis
is dU/dr. 2. x-axis is separation r, y-axis is absolute error on a log scale. Save
`artifacts/ad/modes.png`; print each maximum error.
- sample: 601 equally spaced r values from 0.95 to 2.5, including endpoints
- finite difference: h = 1e-6
```

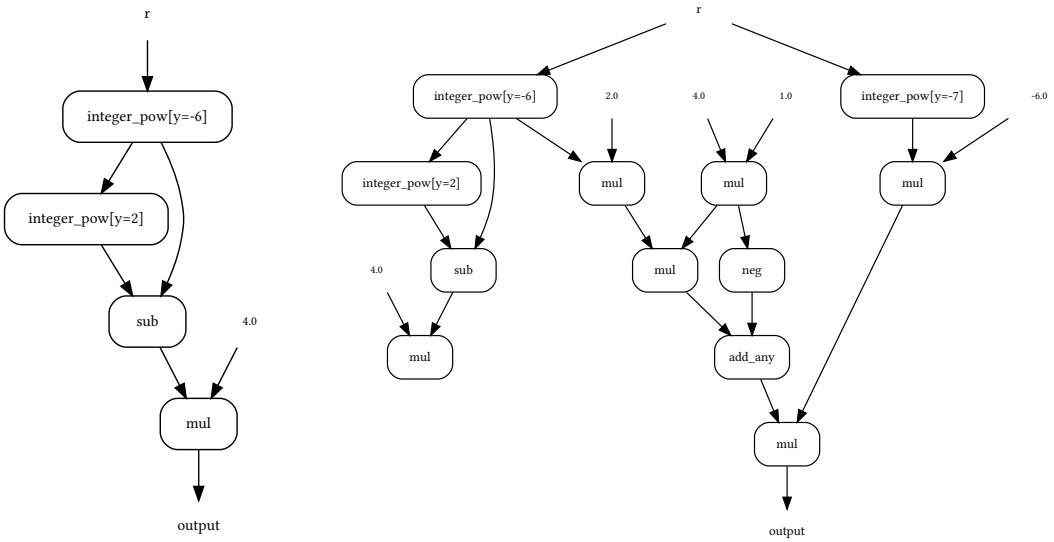
Open `artifacts/ad/modes.png`. The curves must follow Equation 6 and cross zero at $r = 2^{1/6} \approx 1.1225$. Both AD maximum errors must be below 10^{-12}; the reference gives less than 3×10^{-14}. The finite-difference maximum is about 5×10^{-9}, larger than either AD maximum; individual samples can be more accurate.



(2) **JAX's gradient graph adds two contributions at the shared value.**

> Draw the computational graph JAX records for U at r = 1.3, and the one it records for the gradient of U. Use one node per operation, labelled with the operation's name. Save them as `artifacts/ad/graph.png` and `artifacts/ad/grad-graph.png`.

Open both files beside the references below. `graph.png` shows the four operations of the Lennard-Jones graph in order: the power -6, the square, the subtraction and the multiplication by 4. In `grad-graph.png`, one addition node (JAX labels it `add_any`) joins the two contributions to $\bar{a}$. The gradient graph also computes $-6r^{-7}$ and $2a$ on its forward side: these are local derivatives, evaluated from the primal values r and a, that the reverse side reuses. Its subtraction and multiplication by 4 feed nothing, because the gradient computes U without returning it. Layout and label style may differ.



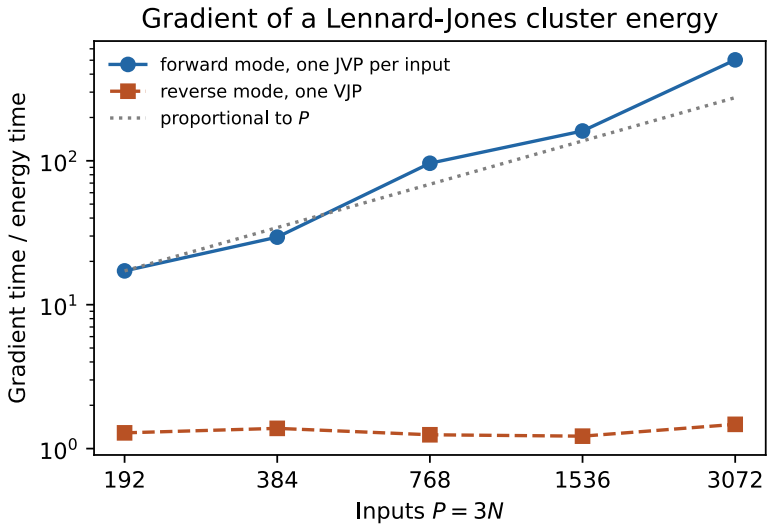
Open `artifacts/ad/derivatives.json`: at $r = 1.3$, require `energy` $= -0.6570169144600471$, `tangents.U`, `adjoints.r`, and `jax_grad` $= 2.239979929791143$, and `adjoints.a` $= -2.3425903117359734$, each within 10^{-12} absolute error. The last value includes both paths through a.

(3) **Reverse mode computes a many-input gradient at the cost of a few energies.**

> Time the gradient of a Lennard-Jones cluster energy with respect to all 3N atom coordinates. Use forward mode, one input direction at a time, and reverse mode. Plot gradient time over energy time against the number of inputs, on log-log axes. Save `artifacts/ad/scaling.png`. Print both ratios and each mode's largest relative error against the analytic forces.
> - cluster: N atoms of a cubic lattice with spacing 2^(1/6), each coordinate moved by a Gaussian of standard deviation 0.05

```
    - sizes: N = 64, 128, 256, 512, 1024
    - relative error: largest absolute error over the largest force component
```

Open `artifacts/ad/scaling.png` beside the reference. Both errors must be below 10^{-12}; the reference gives 2×10^{-15}. The forward curve rises along the dotted line proportional to P, while the reverse curve stays flat: the reference reverse ratio stays between 1.2 and 1.5, and the forward ratio reaches about 500 at $P = 3072$. Your timings will differ, but at $P = 3072$ the forward ratio must exceed 100 times the reverse ratio. One JVP costs less than an energy evaluation here; the check is the growth with P, not that constant.



Part 2: Seismic wave propagation

Goal: receiver traces matching the answer key's amplitudes, and a recording in which the thin reflector sends an echo back toward the receivers.

① DO

Fetch the week's fixed experiment files into `inputs/`. This Part uses the thin-reflector survey; Part 4 uses the Marmousi model, a geological benchmark it describes.

```
curl -fLO https://giggleliu.github.io/AMAT5315-2026Fall/downloads/week5-inputs.zip
unzip -o week5-inputs.zip && rm week5-inputs.zip
```

Create `seismic.design.toml` in `week5/` from this green panel. Start with `forward`; Part 3 adds the two derivative modes.

seismic.design.toml

```
usage       = "seismic --experiment inputs/reflector.json --mode forward --every 3 --
out artifacts/forward"
description = "simulate acoustic waves and their derivatives; no unstated defaults"

[args]
experiment = """JSON file with the model and the survey, in reduced units:
    nx, nz            grid points along x and along depth z
    dx, dt, steps     grid spacing, timestep, number of time steps
    source_frequency  peak frequency f0 of the Ricker pulse
    source_peak_time  time t0 of the pulse's peak
    source_amplitude  source strength
    shots             [x, z] grid index of each shot
    receivers         [x, z] grid index of each receiver
    sponge_width      sponge width in cells
    sponge_strength   largest damping, at the edge
    background        wave speed c, a [z][x] array
    perturbation      velocity perturbation, a [z][x] array, used by born
                      and by the forward recording's echo
    length_unit_m     metres per length unit
    time_unit_s       seconds per time unit"""
mode = "forward: pressure traces; born: Born scattered data; adjoint: migrated image"
out  = "output folder"

[writes]
stdout = "a header, then one tab-separated line per shot: shot, mode, data L2 norm"

"<out>/run.json"    = """experiment_file: the experiment's path;
    experiment: its fields except background and perturbation"""
"<out>/result.json" = "mode, nx, nz, dx, dt, steps, shots, receivers"

"<out>/traces.npy"    = "forward: float64 pressures [shot, step, receiver]"
"<out>/born_data.npy" = "born: float64 scattered data [shot, step, receiver]"
"<out>/image.npy"     = "adjoint: float64 velocity sensitivity [z, x]"

[args.recording]
every = "forward and adjoint only: frame interval in steps from step 0; default off"

[writes.recording]
"<out>/wavefield.npy" = """float32 frames of the first shot [frame, z, x]:
    forward: u^n after step n, n increasing;
    adjoint: the adjoint of u^n after reversing step n, n decreasing"""
"<out>/echo.npy" = """forward only: float32 frames of the first shot [frame, z, x],
    u^n in background plus perturbation minus u^n in the background,
    at the wavefield's frames"""
"<out>/run.json" = """adds recording: every, and steps and times (step*dt),
    one entry per frame in frame order"""
```

```
Brainstorm a Rust `seismic` package following `seismic.design.toml`. Build `forward`
mode only for now.
- equation: u_tt + 2*sigma*u_t = c^2*Laplacian(u) + q
- discretization: centered time differences and a five-point spatial Laplacian
- initial conditions: both stored fields u^-1 and u^0 are zero; no half-step start
- boundary conditions: outer grid values stay zero
- sponge: sigma = sponge_strength * max(0, 1-distance/sponge_width)^2; distance is in
cells from the nearest edge
- q: source_amplitude times the Ricker pulse at each step's start, times a unit-peak
Gaussian footprint of standard deviation one cell, centered at the shot
- receivers: sample after each update
```

Follow the planning and implementation skills from Week 2, the instruction packages your agent installed then, through design, plan, and test-first execution.

(2) UNDERSTAND

The program and its files. A *binary* is an executable program built from a Rust package; *Cargo*, Rust's build tool, compiles it and `cargo install --path .` puts it on your command path. `stdout` is its terminal output. The panel's `born` and `adjoint` modes are built in Part 3, which defines them.

The experiment. The experiment uses a square of side 4 km with 41 points per side. The background speed is 1.8 km/s. Three shots lie at depth 0.8 km. Fourteen receivers share that depth. The forward run here uses the uniform background. The thin reflector becomes the velocity perturbation in Part 3. Its *echo*, the wave it scatters back, is weak: the reflector changes the speed by at most 0.15 km/s, so the echo reaches only about 2–4 percent of the direct wave's largest pressure and cannot be seen in the full field. The recording therefore also runs the first shot in background plus perturbation and keeps the difference from the background run, which is the echo alone.

Acoustic dynamics. The damped acoustic wave equation adds a friction-like term to Equation 1:

$$\partial_t^2 u + 2\sigma\partial_t u = c^2\nabla^2 u + q. \tag{8}$$

Here $\sigma(x, z)$ is the damping coefficient. Damping reduces the wave amplitude as the wave travels. The damping term also takes a centered time difference, $\partial_t u \approx (u^{n+1} - u^{n-1})/2\Delta t$. With the discretization of Equation 3, solving Equation 8 for u^{n+1} gives the update

$$u^{n+1} = \frac{2u^n - (1 - \sigma\Delta t)u^{n-1} + \Delta t^2(c^2 L_h u^n + q^n)}{1 + \sigma\Delta t}. \tag{9}$$

Products and divisions act point by point. The outer grid points stay zero. Both starting fields are zero for the seismic shots.

A sponge damps waves near the edges to reduce artificial reflections. At distance ℓ grid cells from the nearest edge, it uses $\sigma = \sigma_{\max}\max\left(0, 1 - \frac{\ell}{W}\right)^2$, with width $W = 6$ cells and $\sigma_{\max} = 0.4$. This is the course's boundary treatment; Symes uses a perfectly matched layer, a different absorber with additional evolving fields.

Pulse and observations. A Ricker pulse sets the source's time profile $g(t) = (1 - 2\theta^2)\exp(-\theta^2)$, where $\theta = \pi f_0(t - t_0)$, f_0 is the peak frequency and t_0 the peak time. Its Gaussian footprint spreads the source around its grid position (x_s, z_s): $\exp\left(-\left((x - x_s)^2 + (z - z_s)^2\right)/2\right)$, with coordinates in grid cells. Its standard deviation is one grid cell. The experiment file fixes its amplitude, frequency, peak time, source positions and receiver positions. At step n, evaluate the pulse at $t = n\Delta t$. Receivers sample the new field at $t = (n + 1)\Delta t$ after the update.

The files use *reduced units*: one length unit is 100 m and one time unit is 0.1 s. One speed unit is therefore 1 km/s. Here $\Delta x = 1$, $\Delta t = 0.2$, $f_0 = 0.08$, $t_0 = 15$, and there are 240 updates. Coordinates in an experiment are grid indices, in the order $[x, z]$. Field arrays are indexed by depth first, $[z, x]$. The program writes its arrays as `.npy` files: `float64` keeps sixteen digits; `float32`, used for recordings that are only viewed, keeps seven.

The L2 norm $\|u\|_2$ of an array is the square root of the sum of its squared entries.

KEY CONCEPTS

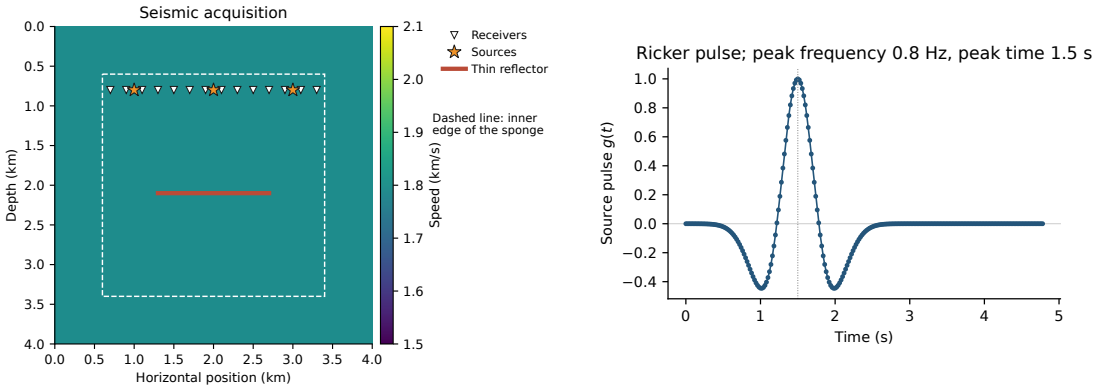
- The damped acoustic wave equation adds damping to the wave equation; centered time differences and the five-point spatial Laplacian give one update per step.
- A sponge raises the damping near the edges to absorb outgoing waves.
- A Ricker pulse times a Gaussian footprint is the source; receivers sample the new pressure after each update.
- Reduced units make one length unit 100 m and one time unit 0.1 s.
- The reflector's echo is a few percent of the direct wave; the difference of a run with and without the reflector isolates it.
- The L2 norm summarizes all traces in one number.
- **The check:** compare the waveforms, then watch the reflector's echo.

3 VERIFY

(1) **The experiment file places receivers above the reflector and defines a Ricker pulse.**

```
Plot what `inputs/reflector.json` defines, in two panels, and save `artifacts/
inputs.png`.
- geometry: the background speed, the perturbation, the shots and the receivers;
horizontal position in km on the x-axis, depth in km increasing down the y-axis
- source pulse: the Ricker pulse at each step's start, against time in seconds
```

Open `artifacts/inputs.png` beside these figures. The geometry shows three shots and fourteen receivers at 0.8 km depth over a uniform 1.8 km/s background, the reflector at 2.1 km. The pulse peaks at 1 at $t_0 = 1.5$ s between two troughs of -0.446 at 1.02 s and 1.98 s, and is negligible after 2.6 s of the 4.8 s run. Its peak frequency $f_0 = 0.08$ reduced units is 0.8 Hz.

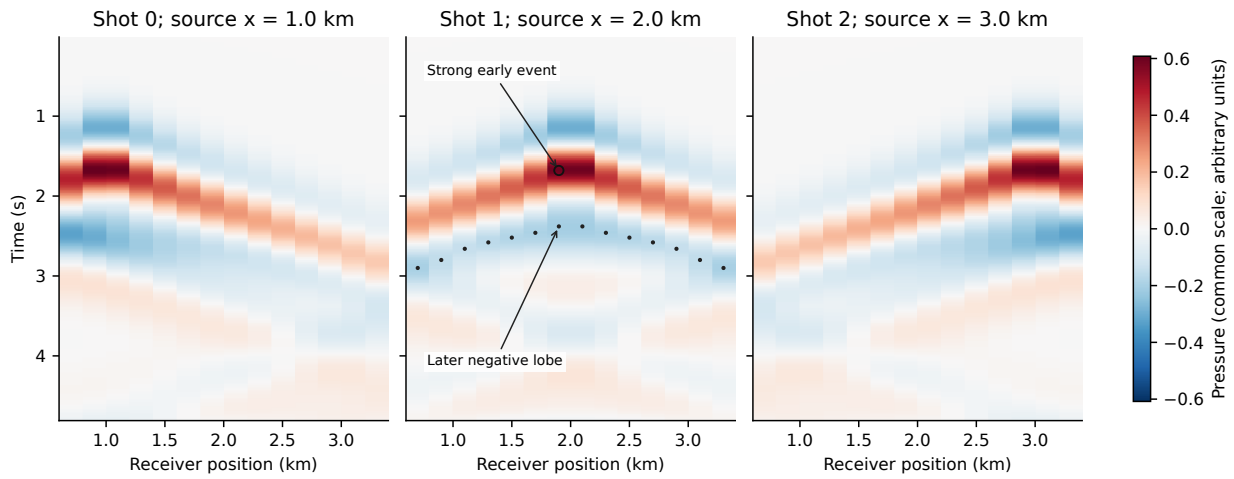


(2) **The receiver waveforms match the reference amplitudes.**

```
cargo install --path seismic
seismic --experiment inputs/reflector.json --mode forward --every 3 --out \
  artifacts/forward
```

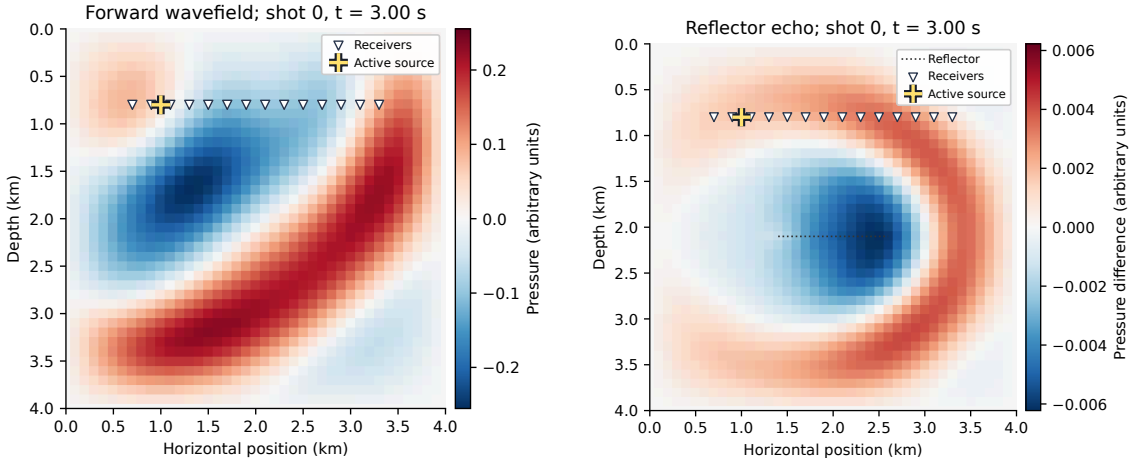
```
Plot the forward run's receiver traces as one shot gather per shot. Put receiver
position in km on the x-axis, time in seconds increasing down the y-axis, and
pressure as colour. Save `artifacts/forward/gathers.png`. Print the L2 norm of all
traces, and each shot's largest absolute pressure with its trace index and
receiver.
```

Open `artifacts/forward/gathers.png` and read the printed values. The reference L2 norm of all traces is 11.574770; require relative error below 10^{-4}. Each shot's largest pressure occurs at trace index 83, after step 84 ($t = 16.8$ reduced units), at a receiver one cell from its source: 0.60809514 for the outer shots and 0.59271397 for the central one, also within relative error 10^{-4}. The early strong event lies near the source; the later negative lobe curves toward larger times at distant receivers.



(3) **The reflector's echo appears in the difference of two runs.**

Open the course viewer and drop `artifacts/forward/wavefield.npy` and `artifacts/forward/run.json` onto it. Play the recording: the pulse spreads from the first shot, and nothing marks the reflector at 2.1 km depth. Select step 150 at 3.00 s and save `artifacts/forward/wavefield.png` using **Save PNG**. Then drop `artifacts/forward/echo.npy` with the same `run.json` and play it. The frames stay blank for about 1.5 s, until the direct wave reaches the reflector; then a scattered wave spreads from the reflector back toward the receivers. Save step 150 as `artifacts/forward/echo.png` and compare both frames with these. The echo's largest value, about 0.006, is about 2 percent of the direct wave's largest pressure, 0.26, at the same step, which is why the full recording hides it.



Part 3: Differentiating the seismic simulation

Goal: a reverse pass that passes the transpose check against the forward pass, and the image it forms, which locates the thin reflector within one grid cell of its true depth.

① DO

Install the course's Enzyme setup skill, which pins the compiler and isolates the differentiated code:

```
Install the `enzyme-setup` skill from
https://giggleliu.github.io/AMAT5315-2026Fall/downloads/enzyme-setup/SKILL.md into
this repository's skills folder.
```

Start a fresh session so the new skill loads, then resend Part 1's session-rules panel before continuing. Append this panel to `seismic.design.toml`:

`seismic.design.toml`

```
[args.reverse]
data = "born_data.npy of a born run: the receiver weights to back-propagate"

[writes.reverse]
"<out>/result.json" = """adds a statistics object:
    storage                 "full": every complete state is kept
    checkpoints             null
    reverse_calls           timestep VJPs
    scheduler_forward_calls forward steps run to supply saved states,
                            not the step repeated inside each VJP
    peak_saved_states       most complete states held at once, N+1 here
    peak_saved_bytes        their size; one state is 2*nx*nz*8 bytes
    per_shot                per shot, in shot order: reverse_calls,
                            scheduler_forward_calls, peak_saved_states;
                            the flat counters above aggregate these"""
```

```
Add `born` mode, the Born approximation, to `seismic` as `seismic.design.toml`
specifies. Use the Enzyme setup skill. Chain Enzyme's timestep JVPs to propagate the
experiment's velocity perturbation. Hold the initial state, source and damping fixed.
```

```
Add `adjoint` mode to `seismic`, as `seismic.design.toml` specifies. Store the full
trajectory, then chain Enzyme's timestep VJPs backward through time. Back-propagate
the receiver weights given by `--data`. Sum the velocity sensitivity over time and
shots.
```

(2) UNDERSTAND

Enzyme and the compiler. Enzyme generates derivative code while Rust compiles the numerical calculation. An *active* input is differentiated; a *constant* input is held fixed. The Enzyme setup skill is an instruction package for your agent. It pins a *nightly* compiler, a daily development build of Rust that carries Enzyme. It isolates the differentiated code from the rest of the program, which avoids a compiler failure in that nightly. Use the course Linux machine if your platform cannot run this setup.

Forward and reverse mode through time. Part 3 computes the opening's Born data $d = Jm$ with forward mode and its image $m_{\text{img}} = J^T d$ with reverse mode. The simulation is a chain of N steps, $s_{n+1} = H_n(s_n, c)$: Part 1′s computational graph with one node per time step. Let A_n and B_n be the step's derivatives with respect to the state and the velocity, and let R sample the receivers from a state. Its transpose R^T places each receiver weight back at its grid location, adding weights if locations repeat. Enzyme computes each step's JVP and VJP; the Rust loop chains them as Part 1′s boxes chain the nodes. A receiver weight w_n assigns a coefficient to each pressure sampled after step n. The velocity sensitivity m_{img} accumulates, cell by cell, how the weighted data change with the velocity:

function BORN-FORWARD(m)
  $\dot{s}_0 \leftarrow 0$ ▷ initial fields are fixed
  for $n = 0, 1, \ldots, N-1$ **do**
    $\dot{s}_{n+1} \leftarrow A_n \dot{s}_n + B_n m$ ▷ one JVP
    $d_n \leftarrow R \dot{s}_{n+1}$ ▷ sample the receivers
  end
  return $d = Jm$
end

function MIGRATION-REVERSE(w)
  $s_0, \ldots, s_N \leftarrow$ the forward run ▷ keep the full trajectory
  $\overline{s}, m_{\text{img}} \leftarrow 0, 0$
  for $n = N-1, \ldots, 1, 0$ **do**
    $\overline{s} \leftarrow \overline{s} + R^T w_n$ ▷ the receivers re-emit
    $m_{\text{img}} \leftarrow m_{\text{img}} + B_n^T \overline{s}$ ▷ one VJP, needs s_n
    $\overline{s} \leftarrow A_n^T \overline{s}$
  end
  return $m_{\text{img}} = J^T w$
end

Recorded frame by frame, $\overline{s}$ is a time-reversal mirror: the receivers re-emit the data and the wave runs backwards until it converges on the structure that scattered it.

The dot-product identity. The dot-product identity, also called the transpose identity,

$$\square Jm, w\square = \square m, J^T w\square, \tag{10}$$

checks the two passes against each other. Both inner products are ordinary sums over array entries. With the Born data as weights, $w = d = Jm$, the left side is $\|d\|_2^2$ and the right side pairs the perturbation with the image.

The image's depth profile. The row L2 norm of an image is the L2 norm of one row, a fixed depth z; plotted against z, it peaks where the image is strongest. For array comparisons, the relative L2 error is $\|u - u_{\text{ref}}\|_2 / \|u_{\text{ref}}\|_2$, with the reference array u_{ref} stated by each check.

Storage for the reverse calculation. Each VJP needs its primal input state s_n. The full trajectory contains all $N+1$ states of an N-step simulation; for $N = 240$, that is 241 complete two-field states. Keeping it all is full history. Part 4 replaces it with a checkpoint schedule.

- Read Symes [2], "Example: reverse time migration," equations 3–5. Find the two fields in a state, the factors of the rule that forms the image (here $m_{\text{img}} \leftarrow m_{\text{img}} + B_n^T \overline{s}$), and where the adjoint becomes the gradient of a least-squares misfit.
- Read Liu and Xu [1], its seismic differentiation discussion. Identify where a local chain rule becomes a derivative of an entire trajectory, and which factors of the update a plain product of forward and adjoint pressures omits.

KEY CONCEPTS
- Enzyme, set up by its skill, differentiates one time step; the Rust loop chains the steps.
- Born data are the JVP Jm, one JVP per step.
- A reverse sweep turns receiver weights into the velocity sensitivity $J^T w$, one VJP per step.
- The dot-product identity with $w = Jm$ equates $\|Jm\|_2^2$ and $\square m, J^T Jm\square$.
- Full history keeps the full trajectory, the primal state each reverse step needs.
- The row L2 norm of the image peaks at the reflector's depth.
- **The check:** verify the transpose identity, then locate the reflector in the image and watch the adjoint field refocus on it.

3 VERIFY

(1) **The image satisfies the transpose identity and locates the reflector.**

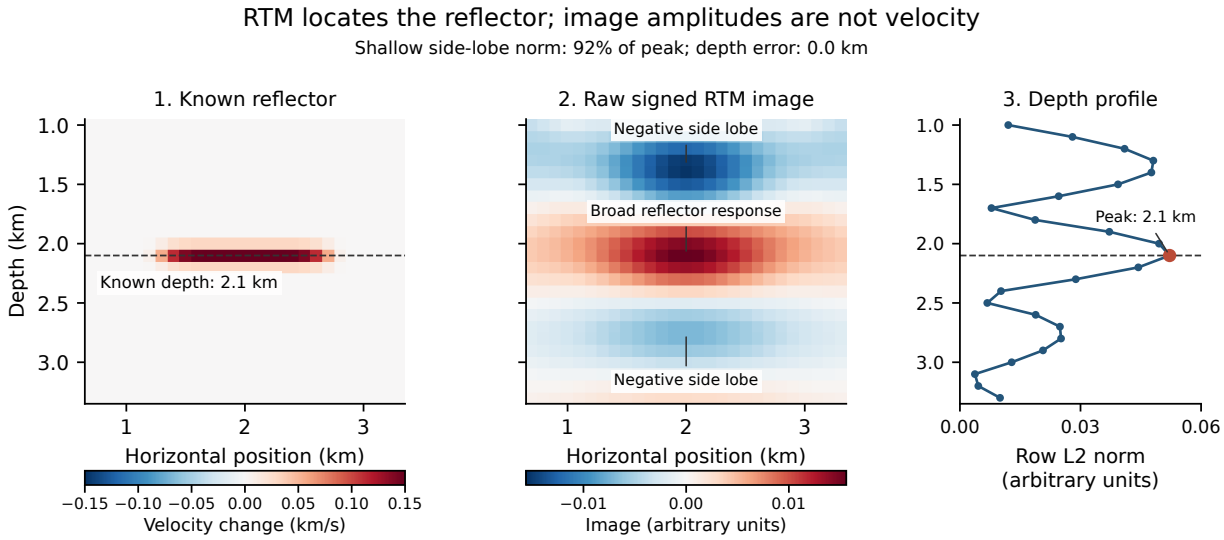
```
cargo install --path seismic
seismic --experiment inputs/reflector.json --mode born --out artifacts/born
seismic --experiment inputs/reflector.json --mode adjoint --data \
  artifacts/born/born_data.npy --every 3 --out artifacts/adjoint
```

```
Evaluate the dot-product identity. Print both sides and their relative difference.
- left: sum of squared entries of the Born data
- right: sum over cells of the perturbation times the image
- relative difference: abs(left-right)/max(abs(left),abs(right))
```

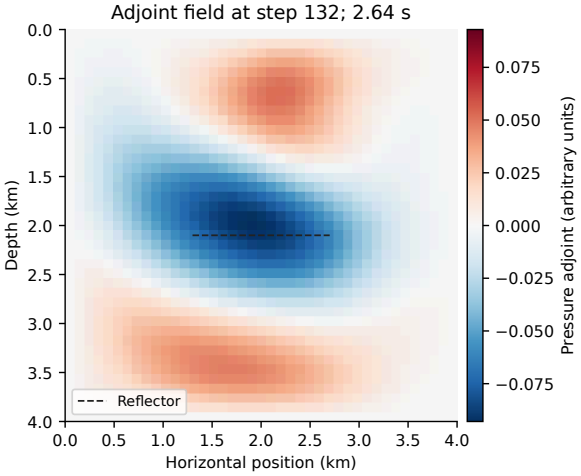
Read the printed values. The reference gives $\|d\|_2^2 = 0.0348479$ on both sides. Require the two sides to have relative difference below 10^{-9}; the reference gives 10^{-15}. A mismatch means the operators disagree: check the JVP, reverse propagation, receiver injection and shared velocity parameterization.

> Draw three panels over grid x=7..33, z=10..33 inclusive: 1. Perturbation; x-axis is horizontal position in km, colour is velocity perturbation in km/s. 2. Image; x-axis is horizontal position in km, colour is raw RTM amplitude in arbitrary units. 3. Image depth profile; x-axis is row L2 norm in arbitrary units. All y-axes show depth in km increasing downward. Mark the true reflector depth and the profile's peak; print their depth difference. Use separate colour scales. Save `artifacts/adjoint/image.png`.

Open `artifacts/adjoint/image.png`. The profile's largest value in the stated window must lie within one cell of $z = 21$, or depth 2.1 km; the window excludes the rows near the sources and receivers. The reference peaks at 2.1 km, exactly the input depth. The broad positive band locates the reflector; the surrounding negative bands are side lobes of this single reflector's image. Finite source bandwidth and limited source–receiver coverage blur the response. RTM recovers the location here, while its amplitudes and width differ from the input velocity perturbation.



Open the course viewer and drop `artifacts/adjoint/wavefield.npy` and `artifacts/adjoint/run.json` onto it. Played in order, the frames run backwards in time: the receivers re-emit the scattered data and the adjoint field converges onto the reflector. Select step 132, at $t = 2.64$ s, and use **Save PNG** to save `artifacts/adjoint/wavefield.png`; compare with this frame.



Part 4: Treeverse checkpointing

Goal: a checkpointed reverse pass that reproduces the full-history image with six saved states, a measured storage–recomputation curve, and a Marmousi image made with six saved states where full history would need 4.16 GB per shot.

① DO

Append the final panel to `seismic.design.toml`:

```
seismic.design.toml
[args.checkpoint]
storage     = "how adjoint keeps states: full (default) or treeverse"
checkpoints = "treeverse only: saved-state slots beyond the initial state"

[writes.checkpoint]
"<out>/actions-<shot>.json" = """one shot's schedule, one entry per action,
    each with action, step n, and saved_states (count after it, s_0 included):
    store    save the working state s_n in a new slot
    restore  copy the saved s_n into the working state
    call     advance the working state from s_n to s_(n+1)
    grad     reverse step n, reading s_n from a saved slot
    fetch    free the slot holding s_n
    s_0 is saved from the start and never freed;
    steps N-1, ..., 0 are each reversed once, in that order;
    at the end, only s_0 is saved"""

"<out>/result.json" = """statistics object as for full history,
    with storage "treeverse", checkpoints the budget,
    and per_shot adding actions_file, a name in <out>"""
```

```
Add Treeverse checkpointing to `seismic` as `seismic.design.toml` specifies. Port
the Treeverse algorithm of https://github.com/GiggleLiu/TreeverseAlgorithm.jl (`src/
treeverse.jl`) to Rust. Save, restore and replay complete states within the checkpoint
budget. Record the schedule's forward steps and each shot's action file.
```

Start these runs, about five minutes in all. Part 2's archive included `inputs/marmousi.json` and its license; never run Marmousi with full history.

```
cargo install --path seismic
for b in 1 3 5 10; do
  seismic --experiment inputs/reflector.json --mode adjoint --data \
    artifacts/born/born_data.npy --storage treeverse --checkpoints $b \
    --out artifacts/checkpoint-$b
done
seismic --experiment inputs/marmousi.json --mode born --out artifacts/marmousi-born
seismic --experiment inputs/marmousi.json --mode adjoint --data \
  artifacts/marmousi-born/born_data.npy --storage treeverse --checkpoints 5 \
  --out artifacts/marmousi-image
```

② UNDERSTAND

The storage problem. Checkpointing saves selected states and recreates the others by *recomputation*, running the forward update again. A checkpoint budget limits the saved states. Our convention reserves the initial state separately: δ extra slots permit at most $\delta + 1$ saved complete states. A save copies a complete state into a slot. A restore copies it back into the working state. Replay advances that state to the time the reverse step needs. Replay adds nothing to the image; only the reverse calls accumulate the velocity sensitivity. A saved state also carries its time index, so replay uses the correct source pulse.

Treeverse construction. Treeverse constructs a binomial checkpoint schedule for a known number of equal-cost steps and a fixed saved-state budget. Its *repetition number* τ is the largest number of times the scheduler advances any one step. The binomial capacity

$$\beta(\delta, \tau) = \binom{\delta + \tau}{\delta} \tag{11}$$

is the longest simulation δ extra slots can reverse with repetition number τ. Choose the smallest positive τ with $\beta(\delta, \tau) \geq N$. On the steps $n_1, \ldots, n_2 - 1$, written $[n_1, n_2)$, the construction first chooses the split $\kappa = \left\lceil \frac{\delta n_1 + \tau n_2}{\delta + \tau} \right\rceil$, where $\lceil \cdot \rceil$ rounds up. Save the state at κ. Reverse the suffix $[\kappa, n_2)$ with one fewer available slot. Then free the suffix checkpoint at κ, restore n_1, and handle the prefix $[n_1, \kappa)$. The recursive construction repeats the split with a smaller repetition allowance for each successive prefix. The prompt points the agent to *TreeverseAlgorithm.jl*, a Julia implementation of this construction, which fixes the precise endpoint rule and base case: each call finishes by reversing its first step, using that step's saved input. The replay workspace and the AD callback's scratch are separate from saved slots; a replayed state must occupy a slot before it can supply a reverse step.

For $N = 6$ and $\delta = 2$, $\tau = 2$ because $\binom{4}{2} = 6$. The first saved interior state is s_3. The suffix saves s_5 and reverses step 5. It then restores s_3 to reconstruct s_4. After reversing the suffix, the schedule returns to s_0 for the prefix. It uses at most three saved states. The full schedule makes eight forward advances and six reverse calls. This particular split rule need not minimize total forward work for every N and budget; the exercise checks its image, storage and measured work, without an optimality proof.

Work, memory and actions. The forward steps the schedule runs measure its work: full history runs each of the N steps once, and a smaller budget replays some steps again. (Enzyme's reverse call also recomputes its own step inside each VJP; that cost is the same for every schedule, so the count leaves it out.) The peak saved states is the largest number of complete states held at once; shots run one after another, so peaks are maxima over shots and work sums over shots. An action file lists the schedule, one line per action: `store` saves a state, `restore` copies one back, `call` advances one step, `grad` reverses one step, and `fetch` frees a slot.

The Marmousi model. *Marmousi* is a synthetic geological velocity model with curved and dipping layers, a faulted zone and a fast basement. Following Symes, we smooth it to obtain the background c_0 and use its shorter-scale structure as the *reflectivity* m. Our copy is the licensed model distributed with ADSeismic, a Julia package for differentiable seismic simulation [3], 20.1 km wide and 6.7 km deep. Gaussian smoothing with a standard deviation of three cells gives c_0; the original minus a model smoothed over one cell gives m. A 5 Hz source resolves its layers, so the grid is 25 m, 805×269 points, and each shot runs 1200 steps of 0.003 s. One complete state now takes 3.46 MB: six states hold 20.8 MB, while the full history of one shot, 1201 states, would hold 4.16 GB. Nine shots 2 km apart and 91 receivers at 0.1 km depth *illuminate* the upper layers: they send and record enough wave energy there to image them. The deep layers, which the waves barely reach, and the artifact arcs just below the sources show the limits of that illumination. The raw image is $J^T J m$ before any *display gain*, a depth-dependent boost applied for viewing; keep its amplitude scale fixed across depth, or a gain could hide weak illumination.

- Read Griewank [4], the binomial checkpoint construction. Trace a saved state through one suffix and one prefix.
- Read Symes [2], its checkpointing discussion, Table 1 and Figures 1–4: the storage–recomputation tradeoff, then the background, perturbation, shot gather and migrated image. Griewank and Walther [5] describe the Revolve implementation; Zhu et al. [3] go on to source recovery and velocity inversion.

KEY CONCEPTS

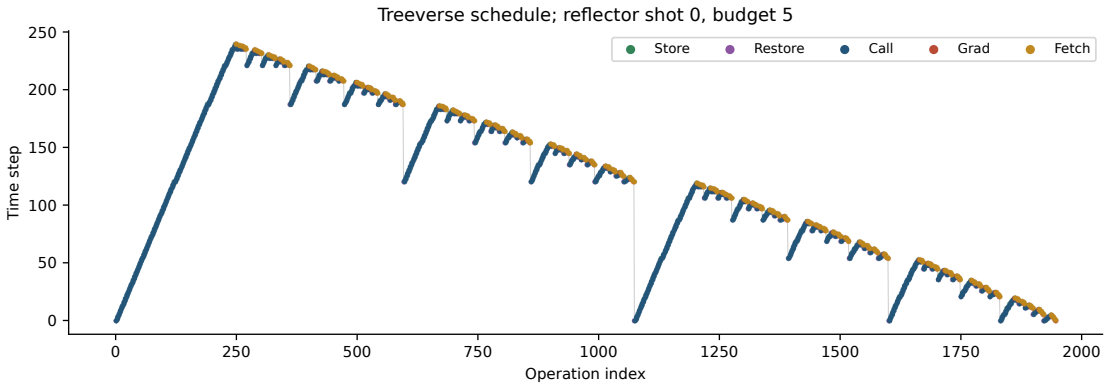
- Checkpointing keeps a few complete states within a checkpoint budget; save, restore and replay supply the others.
- Treeverse's recursive construction assigns the budget.
- A smaller budget costs more forward steps and lowers the peak saved states, while the relative L2 error of the image stays zero.
- An action file lists every store, restore, call, grad and fetch.
- On Marmousi, six states replace 4.16 GB of full history; the raw image loses strength with depth.
- **The check:** compare the checkpointed image with full history, read the action plot and the work curve, then find the dipping layers in the Marmousi image.

③ VERIFY

(1) **Checkpointing reproduces the image with valid replay.**

```
Compare each checkpointed image with the full-history image. Print each relative L2
error and each run's peak saved states. Audit every action file and print three
counts: grad steps that differ from N-1, ..., 0 at their position, including
missing or extra entries; invalid restores; and budget overruns. Plot the first
shot's actions for budget 5, with operation index on the x-axis, timestep on the y-
axis and colour by action type. Save `artifacts/checkpoint-actions.png`.
```

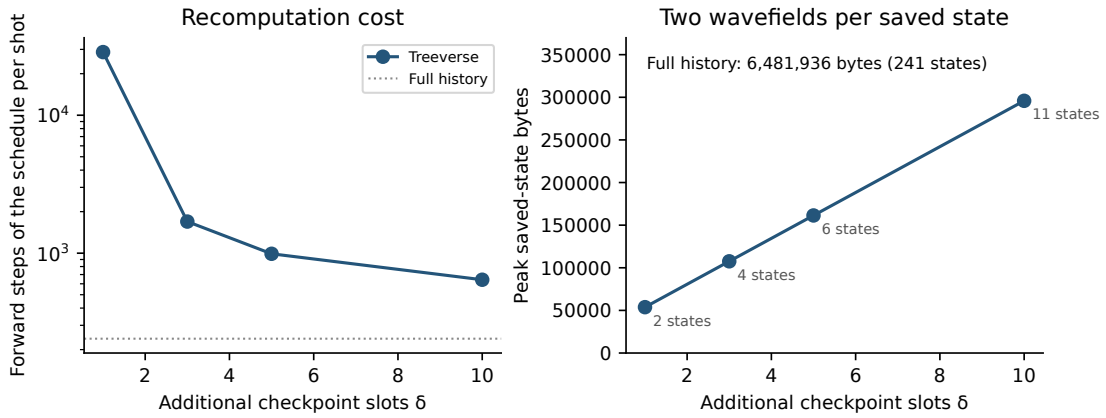
Read the printed values. Each relative error must be below 10^{-9}; the answer key gives zero. Peaks for budgets $1, 3, 5, 10$ are $2, 4, 6, 11$ states. Each shot must reverse steps $239, \ldots, 0$ once, in order: every audit count must be zero. In `artifacts/checkpoint-actions.png`, the Treeverse sawtooth climbs to the last reverse input, then descends through reverse steps; each restore starts a shorter replay. The numerical audit establishes coverage.



(2) **Saved states reduce recomputation.**

```
Draw two plots: 1. x-axis is checkpoint budget in additional slots, y-axis is
forward steps per shot on a log scale. 2. x-axis is checkpoint budget in additional
slots, y-axis is peak saved-state storage in bytes on a linear scale. Include full
history. Save `artifacts/checkpoint-work.png`.
```

Open `artifacts/checkpoint-work.png`. The reference's forward steps per shot for budgets $1, 3, 5$ and 10 are $28, 680, 1, 695, 990$ and 642; full history uses 240, retaining 241 states or $6, 481, 936$ bytes. The bytes grow with the budget while the work falls.

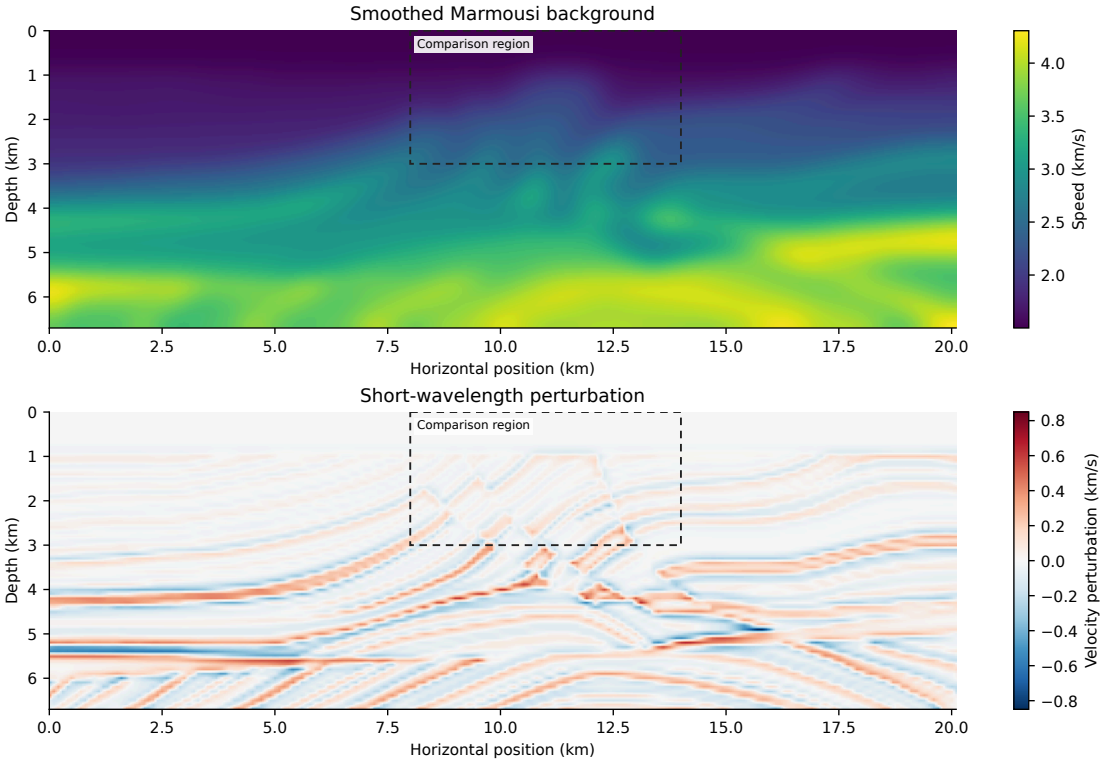


(3) **Six saved states image Marmousi's dipping layers.**

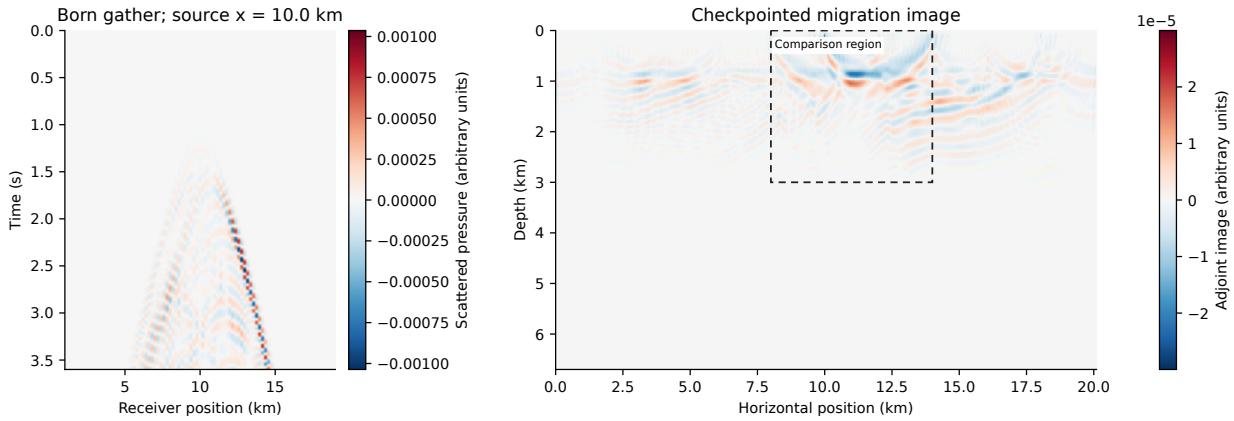
```
Plot four panels of the Marmousi runs in `artifacts/marmousi.png`:
- background
- perturbation
```

```
    - Born data of the shot at x = 10 km, as a shot gather
    - raw image, one amplitude scale across depth
    For the background, the perturbation and the image, put horizontal position in km
    on the x-axis and depth in km, increasing downward, on the y-axis. For the gather,
    put receiver position in km on the x-axis and time in seconds, increasing downward,
    on the y-axis. Print the image's L2 norm and the run's peak saved states and bytes.
```

Open `artifacts/marmousi.png`. Compare its background and perturbation panels with these references: the background is smooth, and curved and dipping layers cross the marked 8–14 km window of the perturbation.



Compare the shot gather and raw image with these references. The image shows inclined bands in the marked upper 3 km window; it need not resolve every fault in the input. Deeper layers remain weak on the same amplitude scale. Require the printed image L2 norm to match 6.7037741×10^{-4} within relative error 10^{-4}, with six saved states holding $20,788,320$ bytes. Explain why an accurate derivative can still produce an image that differs from m.



Final verification: explain the evidence

Defend the derivative checks and image-preserving checkpointing with your evidence. Show the two fields in a restart state, one local Enzyme derivative call, and a replay that leaves the accumulated image unchanged.

The evidence inventory in `week5/` is below; `inputs/` and the `.npy` arrays stay local, and the commands above recreate them:

```
inputs/                 reflector, Marmousi (downloads)
seismic.design.toml     wave-simulation arguments and outputs
src/, scripts/, README.md, and the Rust and Python project files
MARMOUSI-LICENSE
artifacts/
  inputs.png
  ad/                   derivatives.json, modes.png, graph.png, grad-graph.png,
                        scaling.png
  forward/              run.json, result.json, traces.npy, wavefield.npy,
                        echo.npy, wavefield.png, echo.png, gathers.png
  born/                 run.json, result.json, born_data.npy
  adjoint/              run.json, result.json, image.npy, wavefield.npy,
                        wavefield.png, image.png
  checkpoint-{1,3,5,10}/ run.json, result.json, image.npy, actions-<shot>.json
  checkpoint-actions.png, checkpoint-work.png
  marmousi-born/        run.json, result.json, born_data.npy
  marmousi-image/       run.json, result.json, image.npy, actions-<shot>.json
  marmousi.png
```

Your TOML file contains the green panels you wrote as the programs grew. Keep each committed evidence file below 5 MB. The compact `run.json` records the input path; when reproducing, re-download the ignored inputs using Part 2′s command.

Wrap up the work.

> List every evidence file in `README.md`, beside the command or script that produces it. Commit the source, the license and the evidence, but not the `.npy` arrays. Push the exercise repository.

Open each evidence file from the inventory and identify the quantity supporting its claim.

Show the derivative checks, storage–work curves and seismic image to the instructor at the next session. Nothing is submitted. The October 7 holiday leaves two weeks between sessions. The instructor announces the project topics during this week.

Preparation for Week 6. Bring your laptop. The small generative models train on CPU; the image exercise uses downloaded pretrained weights. No cloud account or GPU rental is required.

Bibliography

[1] J.-G. Liu and K.-L. Xu, “Automatic differentiation and its applications in physics simulation,” *Acta Physica Sinica*, vol. 70, no. 14, p. 149402, 2021, doi: 10.7498/aps.70.20210813.

[2] W. W. Symes, “Reverse time migration with optimal checkpointing,” *Geophysics*, vol. 72, no. 5, pp. SM213–SM221, 2007, doi: 10.1190/1.2742686.

[3] W. Zhu, K. Xu, E. Darve, and G. C. Beroza, “A general approach to seismic inversion with automatic differentiation,” *Computers and Geosciences*, vol. 151, p. 104751, 2021, doi: 10.1016/j.cageo.2021.104751.

[4] A. Griewank, “Achieving logarithmic growth of temporal and spatial complexity in reverse automatic differentiation,” *Optimization Methods and Software*, vol. 1, no. 1, pp. 35–54, 1992, doi: 10.1080/10556789208805505.

[5] A. Griewank and A. Walther, “Algorithm 799: Revolve: An implementation of checkpointing for the reverse or adjoint mode of computational differentiation,” *ACM Transactions on Mathematical Software*, vol. 26, no. 1, pp. 19–45, 2000, doi: 10.1145/347837.347846.